Quest 11: The Portalignment

Lord of the Wings · shortcuts across a ring of portals

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: portals sit in a ring and are linked to their neighbours; extra two-way shortcuts cut across the ring. Add up the shortest distance between **every** pair of portals, and report how much the shortcuts reduce that total.
Input: `portals: N`, then one shortcut per line (`a b`).
Output: the *portal resonance* = strain without shortcuts − strain with them.
What changes between parts: Part I links each portal to its 1 nearest neighbour on each side. Part II links the 3 nearest on each side and lets you add **one** extra connection anywhere. Part III uses the Part II ring and adds **four** extra connections, made by 8 crystals spaced evenly round the ring and paired up.

1. The story

Once the Nullghosts have faded, the team goes deeper under Nulldor and finds a huge round chamber with an ancient portal ring, the only way up into the fortress. Gimlock knows how to tune it: measure how much the known shortcuts shrink the total travel distance. Two more rings follow on the levels above, and for those the team can place tuning crystals to make extra shortcuts of their own, as cleverly as possible.

2. Understanding the question

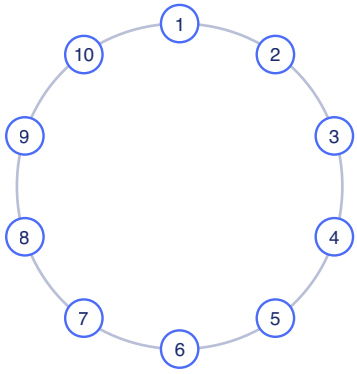
The rules

```
strain(graph) = Σ over every start portal s
                  Σ over every other portal t   (fewest teleports from s to t)

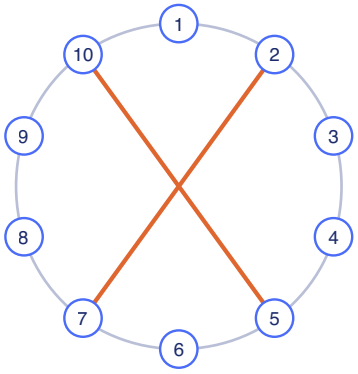
resonance = strain(ring only) − strain(ring + shortcuts [+ crystals])
```

- The distance between two portals is the **number of teleports** on the shortest route: every connection costs 1, whether it's a ring link or a long shortcut. In graph words: an unweighted graph, so **breadth-first search** (BFS) gives exact distances.
- The strain counts every ordered pair, so the trip 1 → 5 and the trip 5 → 1 are both included. That's why the plain 10-ring has strain 25 × 10 = 250, not 125.
- Shortcuts work both ways. A shortcut can never make the strain larger, only equal or smaller.

The sample ring (Parts I and II)



ring only: strain 250



with 5–10 and 2–7: strain 190

Extra examples of my own (run through copies of your Part I code)

Notes	Resonance	Why
10 portals, `1 2`	0	1 and 2 are already neighbours, so the shortcut adds nothing.
10 portals, `1 6`	32	A shortcut straight across the ring: the farthest pair (distance 5) becomes distance 1.
10 portals, `1 6` twice	32	A duplicate line just adds the same connection twice. BFS doesn't care.
6 portals, `1 4`, `2 5`, `3 6`	12	Strain falls from 54 to 42: every portal is now at most 2 teleports from every other one.

Shared code: reading the notes, building the ring, BFS

All three parts share this code. The only difference is the ring itself in `buildGraph` (1 neighbour each side in Part I, 3 in Parts II and III), shown in each part.

```ts
import { readFileSync } from "node:fs";

const isSample = process.argv.includes("--sample");
const file = isSample ? "sample1.txt" : "input1.txt";
const INPUTS: string[] = readFileSync(file, "utf8")
  .trim()
  .split("\n")
  .filter((i) => i);

const TOTAL_PORTAL = Number(INPUTS[0]!.split(" ")[1]);

INPUTS.shift();

const SHORTCUTS: [number, number][] = INPUTS.map((line) => {
  const [a, b] = line.trim().split(/\s+/).map(Number);
  return [a!, b!];
});
```

q11/part1.ts, lines 1–17

```ts
const routeSum = (graph: number[][], start: number): number => {
  const dist = new Array<number>(TOTAL_PORTAL + 1).fill(-1);
  dist[start] = 0;
  const queue = [start];
  let sum = 0;

  for (let head = 0; head < queue.length; head++) {
    const current = queue[head]!;

    for (const next of graph[current]!) {
      if (dist[next] !== -1) continue;

      dist[next] = dist[current]! + 1;
      sum += dist[next]!;
      queue.push(next);
    }
  }

  return sum;
};

const strain = (graph: number[][]): number => {
  let total = 0;
  for (let p = 1; p <= TOTAL_PORTAL; p++) total += routeSum(graph, p);
  return total;
};
```

q11/part1.ts, lines 39–64

Lines	What happens
3–8	`--sample` picks the sample file. Split into lines and drop empty ones (the blank line after the header).
10	`"portals: 10"`.split(" ")[1] → `10`.
12–17	Remove the header line; every other line becomes a pair `[a, b]`. `split(/\s+/)` also copes with several spaces.
39–43	`routeSum`: BFS from `start`. `dist` uses –1 for "not reached yet"; slot 0 is unused because portals start at 1.
45–46	The queue is a plain array read with a `head` index instead of `shift()`, so taking the next portal is O(1).
48–53	Each neighbour seen for the first time is one step further than `current`. Its distance is added to `sum` straight away, so no second pass over `dist` is needed.
60–64	`strain`: run one BFS from every portal and add up the route sums.

Cost: one BFS visits every portal and connection once, so it's O(N + E). The strain runs N of them: O(N · (N + E)), which is O(N^2) here because each portal has only a handful of connections.

3. Part I: the plain ring

The question

Each portal is linked to the one before and the one after it (and N wraps round to 1). Compute the strain without and with the listed shortcuts and print the difference.

Worked examples

From portal	1	2	3	4	5	6	7	8	9	10	Strain
ring only	25	25	25	25	25	25	25	25	25	25	250
with shortcuts	19	17	21	21	17	19	17	21	21	17	190

Resonance 250 − 190 = **60**. Both rows of route sums were printed by an instrumented copy of your code and match the puzzle exactly. The extra examples on the previous page are more Part I cases.

Step-by-step solution

1. Read N and the list of shortcuts.
2. Build the ring as an adjacency list: link every portal to the next one, with N wrapping round to 1.
3. For each start portal, run a BFS and add up the distances to all other portals. The total over all starts is the strain.
4. Do this once for the ring alone and once with the shortcuts added, and print the difference.

My solution

```
19  const buildGraph = (withShortcuts: boolean): number[][] => {
20    const graph: number[][] = Array.from({ length: TOTAL_PORTAL + 1 }, () => []);
21
22    for (let p = 1; p <= TOTAL_PORTAL; p++) {
23      const next = (p % TOTAL_PORTAL) + 1;
24
25      graph[p]!.push(next);
26      graph[next]!.push(p);
27    }
28
29    if (withShortcuts) {
30      for (const [a, b] of SHORTCUTS) {
31        graph[a]!.push(b);
32        graph[b]!.push(a);
33      }
34    }
35
36    return graph;
37  };
```

q11/part1.ts, lines 19–37

```
66  const before = strain(buildGraph(false));
67  const after = strain(buildGraph(true));
68
69  console.log(before - after);
```

q11/part1.ts, lines 66–69

Lines	What happens
20	An adjacency list with N + 1 empty lists (index 0 unused).
22–27	Link `p` to `next`. `(p % N) + 1` gives p + 1, except N → 1, which closes the ring. Each link is pushed in both directions.
29–34	If asked, add every shortcut, also in both directions.
66–69	Build two graphs, measure both, print the difference.

The adjacency list with shortcuts (sample)

Portal	1	2	3	4	5	6	7	8	9	10
Neighbours	2, 10	1, 3, **7**	2, 4	3, 5	4, 6, **10**	5, 7	6, 8, **2**	7, 9	8, 10	9, 1, **5**

Trace: `routeSum(graph, 1)` with shortcuts

`current`	Newly reached (distance)	`sum` after
1	2 (1), 10 (1)	2
2	3 (2), 7 (2)	6
10	9 (2), 5 (2)	10
3	4 (3)	13
7	6 (3), 8 (3)	19
9, 5, 4, 6, 8	nothing new	19

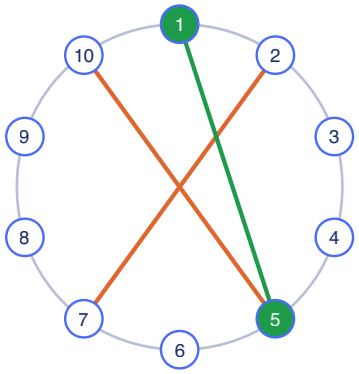
The route sum from portal 1 is **19**, as in the puzzle. Your BFS reaches portal 8 through 1 → 2 → 7 → 8 while the puzzle shows 1 → 10 → 9 → 8. Both are 3 teleports; BFS only cares about the length, not which route.

4. Part II: a wider ring and one crystal pair

The question

Now every portal is linked to the **3** nearest portals on each side. The listed shortcuts are there too, and you may add one more two-way connection between **any** two portals. Choose it to make the resonance as large as possible. The "before" strain is still the ring alone.

Worked examples



sample with crystals at 1 and 5

Configuration	Route sums from portals 1 … 10	Strain
ring only	12 each	120
ring + 5–10 + 2–7		116
… + crystals 1–5	11 11 12 12 10 12 11 12 12 11	114

Resonance 120 − 114 = **6**. Your code tries all 45 pairs; **13** of them reach 114 (1–5, 1–6, 1–7, 2–6, 2–8, 3–7, 3–8, 3–9, 4–8, 4–9, 4–10, 5–9, 6–10) and the other 32 don't help at all (116). The puzzle's "several equally good choices" is right.

Extra examples of my own (copies of your Part II code)

Notes	Answer	Why
10 portals, no shortcuts	2	Strain 120 → 118: one crystal pair across the ring saves a little.
20 portals, `5 10`, `2 7`	64	On a bigger ring, shortcuts and the crystal save far more.
7 portals, no shortcuts	0	With 3 links each side, every portal already reaches all 6 others in one teleport.

Step-by-step solution

1. Build the ring with links to the 3 nearest portals on each side, and measure its strain ("before").
2. Add the listed shortcuts.
3. For every pair of portals a < b: add the connection a–b, measure the strain, keep the smallest, then remove the connection again.
4. Print "before" minus the smallest strain.

My solution

Lines 1–17 and the BFS/strain code (41–66) are the same as Part I (apart from the file names).

```
19  const buildGraph = (withShortcuts: boolean): number[][] => {
20    const graph: number[][] = Array.from({ length: TOTAL_PORTAL + 1 }, () => []);
21
22    for (let p = 0; p < TOTAL_PORTAL; p++) {
23      for (let k = 1; k <= 3; k++) {
24        const next = ((p + k) % TOTAL_PORTAL) + 1;
25
26        graph[p + 1]!.push(next);
27        graph[next]!.push(p + 1);
28      }
29    }
30
31    if (withShortcuts) {
32      for (const [a, b] of SHORTCUTS) {
33        graph[a]!.push(b);
34        graph[b]!.push(a);
35      }
36    }
37
38    return graph;
39  };
```

q11/part2.ts, lines 19–39 (highlighted lines are the ones that changed from the previous part)

```
68  const before = strain(buildGraph(false));
69
70  const graph = buildGraph(true);
71
72  let best = strain(graph);
73
74  for (let a = 1; a <= TOTAL_PORTAL; a++) {
75    for (let b = a + 1; b <= TOTAL_PORTAL; b++) {
76      graph[a]!.push(b);
77      graph[b]!.push(a);
78
79      best = Math.min(best, strain(graph));
80
81      graph[a]!.pop();
82      graph[b]!.pop();
83    }
84  }
85
86  console.log(before - best);
```

q11/part2.ts, lines 68–86 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
22–29	`p` now runs 0 … N – 1 (0-based). For `k` = 1, 2, 3, link portal `p + 1` to portal `((p + k) % N) + 1`, the one `k` places further on, wrapping round. Each link is only made from its lower end, so each appears once in each list.
68	The "before" strain uses the ring alone, as the puzzle says.
70–72	Build the graph with shortcuts once. Start `best` at its strain without any crystal.
74–75	Every pair `a < b`: N(N – 1)/2 candidates.
76–77	Add the crystal connection temporarily.
79	Measure and keep the smallest strain.
81–82	Undo it. `pop()` removes exactly what lines 76–77 pushed, since those were the last items added to the two lists. This avoids rebuilding the graph for every pair.
86	Resonance = ring-only strain – best strain.

Trace: what the loop sees (sample)

Step	Value
neighbours of portal 1 (ring only)	2, 3, 4, 8, 9, 10
`before`	120
`best` at line 72	116
pairs tried	45 (plus line 68 and line 72: 47 strains in all)
first pair that lowers `best`	1–5 → 114
printed	120 – 114 = 6

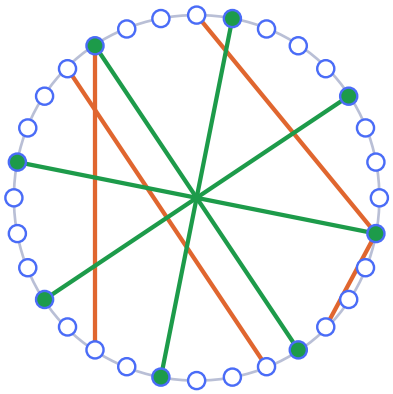
> **Cost:** $N^2/2$ pairs × one strain of $O(N^2)$ each = **$O(N^4)$**. That's fine for the sizes here (your real input runs in well under a second), but it would grow quickly: doubling N makes it about 16× slower.
> **Line 72 is harmless:** the "no crystal" strain isn't a valid choice (the crystals must be placed), but adding a connection never makes the strain bigger, so some pair always ties or beats it.

5. Part III: eight crystals, evenly spaced

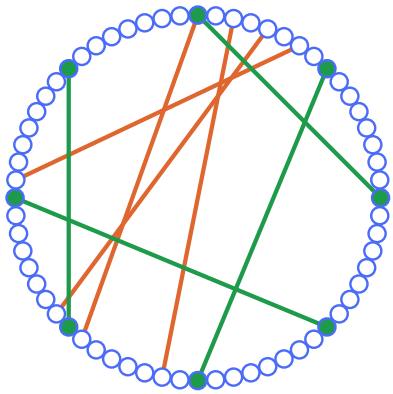
The question

Same ring as Part II, with its listed shortcuts. Now 8 crystals must sit at **equal intervals**: if the first is at portal *s*, the others are at *s* + g, *s* + 2g, …, with g = N / 8. Then pair the 8 crystals into 4 two-way connections. Choose the start *s* and the pairing to maximise the resonance.

Worked examples



32 portals: crystals 2, 6, … , 30



64 portals: crystals 1, 9, … , 57

Red: listed shortcuts. Green: the crystal connections your code found first (filled dots are crystals).

Sample	g	Ring only	+ shortcuts	Best with crystals	Resonance
32 portals	4	3072	2610	2218	**854**
64 portals (sample3.txt)	8	23232	16994	13598	**9634**

The 32-portal example isn't in your sample file, so I ran it through a copy of your code: it gives 854 and finds exactly the puzzle's placement 2, 6, …, 30 with pairs 2–18, 6–22, 10–26, 14–30. For the 64-portal sample it finds start 1 with pairs 1–17, 9–33, 25–49, 41–57. Neither is "opposite pairs": in the 64 case the best pairing is a mix of quarter- and half-turns.

Best strain for each start position (instrumented copy)

Start *s*	1	2	3	4	5	6	7	8
32 portals	2226	**2218**	2228	2228				
64 portals	**13598**	13606	13626	13682	13678	13736	13630	13670

The start really matters, because the listed shortcuts make the ring uneven. Starts beyond g would just repeat these sets of crystals (start *g* + 1 is the same set as start 1), which is why the code stops at g.

Step-by-step solution

1. Build the Part II ring with its shortcuts, and measure the ring-only strain.
2. Work out the gap g = N / 8. For each start s = 1 … g, the crystals are s, s + g, …, s + 7g.
3. Recursively pair the crystals: take the first unpaired one, pair it with each remaining one in turn, and recurse on the rest.
4. When all 4 connections are in place, measure the strain and keep the smallest. Undo each connection on the way back.
5. Print the ring-only strain minus the smallest strain.

My solution

Lines 1–72 are Part II's code (only the file names differ). The new part replaces the pair loop:

```
70  const graph = buildGraph(true);
71
72  let best = strain(graph);
73
74  const CRYSTALS = 8;
75  const GAP = TOTAL_PORTAL / CRYSTALS;
76
77  const tryPairings = (left: number[]): void => {
78    if (left.length === 0) {
79      best = Math.min(best, strain(graph));
80      return;
81    }
82
83    const [first, ...rest] = left;
84    const a = first!;
85
86    for (let i = 0; i < rest.length; i++) {
87      const b = rest[i]!;
88
89      graph[a]!.push(b);
90      graph[b]!.push(a);
91
92      tryPairings([...rest.slice(0, i), ...rest.slice(i + 1)]);
93
94      graph[a]!.pop();
95      graph[b]!.pop();
96    }
97  };
98
99  for (let start = 1; start <= GAP; start++) {
100   const crystals = Array.from({ length: CRYSTALS }, (_, i) => start + i * GAP);
101   tryPairings(crystals);
102 }
103
104 console.log(before - best);
```

q11/part3.ts, lines 70–104 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
74–75	8 crystals, gap g = N / 8.
77–81	`tryPairings(left)`: once every crystal is paired, the graph holds all 4 new connections, so measure it.
83–84	Take the first unpaired crystal `a`. It must be paired with somebody…
86–96	…so try each remaining crystal `b`: add the connection, recurse on the crystals that are still left, then undo with `pop()` (same trick as Part II).
99–102	Try every start 1 … g. `start + i · g` lists the 8 evenly spaced portals.
104	Resonance as before.

How many pairings? (a small example of the recursion)

Always pairing the **first** crystal means each way of pairing is produced exactly once: for 4 crystals A B C D the recursion makes A–B + C–D, A–C + B–D, A–D + B–C, which is all 3. For 8 crystals: 7 × 5 × 3 × 1 = **105** pairings.

Sample	Starts (g)	Pairings each	Strains measured	Cost of one strain
32 portals	4	105	420	BFS from 32 portals
64 portals	8	105	840	BFS from 64 portals

Overall (N / 8) × 105 strains of $O(N^2)$ each, about $13 \cdot N^3$: cheaper than Part II's N^4 once N gets large. Your real input runs in about 0.2 s.

> **Fragile: N must be a multiple of 8.** `GAP = TOTAL_PORTAL / CRYSTALS` (line 75) is not rounded. For a 30-portal ring, g = 3.75, the crystals land on portals like 4.75, and `graph[a]` is `undefined`: a copy of your code crashed with *TypeError: Cannot read properties of undefined (reading 'push')* at line 90. The puzzle's "perfectly equal intervals" only makes sense when 8 divides N, and both examples (and your input) satisfy that, so this is fine for the quest, just worth knowing.
> **Also lucky but harmless:** line 72 again starts `best` at the "no crystals" strain, and duplicate connections (a crystal pair that matches a ring link or a listed shortcut) are simply extra list entries. Both are safe for the same reason as in Part II.

6. Summary

Part	Ring links per side	Extra connections you choose	Search	Strains measured (sample)	Sample answer
I	1	none	two strains	2	60
II	3	1, anywhere	all N(N – 1)/2 pairs	47	6
III	3	4, from 8 evenly spaced crystals	g starts × 105 pairings	842	9634

Key ideas: in an unweighted graph, **BFS** from every node gives all shortest distances (all-pairs shortest paths). When the choices are few enough, simply **try them all**: add an edge, measure, and undo it (backtracking with `push`/`pop`), instead of rebuilding the graph. For pairings, always pair the first remaining item to list every perfect matching exactly once.